cegeka

cegeka
shaping
digital together

Internship SafetySquad

Realization document

Prepared by **Tuur Van Bergen**

Date 1/06/2026

Cegeka nv

Corda 3, Kempische Steenweg 307
B-3500 Hasselt
www.cegeka.com

1 Contact

Van Bergen, Tuur
Internship Student
E: Tuur.VanBergen@cegeka.com
E: R1009675@student.thomasmore.be

Iacob, Franz
Work placement mentor
E: Franz.Iacob@cegeka.com

Cloots, Michaël
Internship Supervisor
E: Michael.Cloots@thomasmore.com

Table of Contents

2 Introduction

This realization document describes the internship conducted by Tuur Van Bergen at Cegeka as part of the SafetySquad project. This project was first introduced at the start of COVID-19 to give Cegeka real-time insights into the attendance and availability of first aid and evacuation personnel across buildings and zones in Belgium.

The application consists of an Angular frontend, a Java backend and two Flutter applications one for mobile and one for desktop. Some further insights into this software can be found in this document.

This internship was an individual assignment focused on the improvement of the existing application, with the emphasis on the creation of a new API. An API, also known as Application Programming Interface, is a component that allows different parts of an application (such as a frontend and backend) to communicate with each other. Alongside the API, several additional features were implemented as described in this document.

The purpose of this document is to describe the features that have been created along with their analysis, the decisions made based on the analysis and the implementation of the feature itself. During the analysis and implementation of these features, the possibility of a future deployment to external companies was considered. This ensured the maintainability and scalability of the application was prioritized.

To better understand the context of this internship, the following chapter provides an overview of the SafetySquad application and its main components.

3 Project Context and Overview

At the start of the COVID-19 pandemic, the workplace model shifted from full-time on-premises work to remote or hybrid work. This transition led to uncertainty regarding the availability of first aid and evacuation personnel across different locations for Cegeka, which resulted in the development of the SafetySquad application.

SafetySquad is a first aid and evacuation registration platform, which gives organizations real-time insight into first aid and evacuation availability across different locations. The application can be used to reduce the manual effort that was currently being conducted to register the attendance of the emergency personnel. Due to this manual tracking of attendees in Excel lists, the data quickly became outdated and unreliable.

With the help of the SafetySquad application and the privacy-by-design approach, attendance can be registered for emergency personnel without intruding on their privacy.

This application consists of four major parts, which are described below.

3.1 Frontend

The frontend application, also known as the website, is responsible for presenting information to the user and enabling interaction with the system. It allows users to view attendance data on the main dashboard and contact the attending emergency personnel in case of an emergency. For emergency personnel, it is also possible to manually check-in through the website to register their attendance.

In addition, the frontend also provides the admin with several useful functionalities, such as managing users and their certificates, as well as creating and managing locations and countries.

For the SafetySquad application, Angular is used. This is a well-known frontend framework maintained by Google. Within Cegeka, this is the most used frontend framework, it will make it easier to maintain by different developers.

3.2 Backend

The backend application, also known as the API, is responsible for handling the requests sent by the frontend. It acts as the connection between the frontend and the database. When a request is made, the API processes it by applying the business logic, such as rules, calculations, and validations. Based on this processing, the backend will retrieve data from or store data in the database. The resulting data will be returned to the frontend where it will be presented to the user.

In the SafetySquad application, Java is used. This is a well-known backend development programming language. Within Cegeka, most long-term projects will use this programming language for backend development as it requires less maintenance. There are also a lot of Java developers within Cegeka which makes it easier for different developers to work on this project in the future.

3.3 Mobile application

The mobile application is responsible for registering a user's presence when they enter or exit a certain location. This is done with the help of geofencing, each location gets assigned a certain geofence, this is a circle with longitude and latitude coordinates and a radius. Once a user enters or exits a certain location, the presence will be registered automatically. This application stays active in the background to handle the presence registration for the user automatically, so there is no need to sign in manually every time.

For the mobile application, Flutter is used. This is an open-source framework developed and maintained by Google that allows developers to build applications for multiple platforms, such as Android and iOS, using a single codebase. This makes the development process and maintenance of the mobile applications easier, as only one mobile application needs to be created and can be used across both platforms.

3.4 Desktop application

Just like the mobile application, the desktop application is also responsible for registering a user's presence when they are at a certain location. Instead of using a geofence we use the IP range of the office building they are in. The application will perform a check of the WIFI network the user is connected to, if it has the correct IP range of a certain office. It will register the user's presence for that location by sending a request to the backend.

Just like the mobile application, the desktop application is also developed using Flutter, because it allows developers to build an application that can be used on both MacOS and Windows and it allows as to keep the amount of programming languages to a minimum. Due to the limited programming languages that are used, less developers are required to maintain this code.

4 Realizations

During my internship, I continued the work started by previous interns, meaning that the overall project scope had already been defined. However, the internship also provided opportunities to perform my own analysis and contribute to the existing project. Since the project had a well-defined scope, most decisions were made following the presentation of research findings and discussions with the work placement mentor. This section describes each feature implemented and the associated research.

4.1 Microsoft Teams Feature

The current version of the application already used a Microsoft Teams implementation; my first task was to research and expand this feature so users could scan a unique QR code for each office location. The QR code implementation itself was straightforward; however, integration with Microsoft Teams required additional research.

For this Microsoft Teams feature, two features were needed:

1. Creation of a group chat with all members present at the location
2. Sending a message in the newly created group

4.1.1 Group chat creation

Creation of a group can be achieved in two ways in the Microsoft Graph API (Microsoft, 2025), the Microsoft Graph API is a service that allows applications to interact with Microsoft tools such as Teams. Permissions determine what actions the application is allowed to perform. Delegated permissions require a user to be logged in, while application permissions allow the system to act independently without user interaction.

Because users do not need to be logged in to scan the QR code, the delegated permissions were not an option. Consequently, application permissions were the only possible solution. After requesting and receiving the necessary permissions from the Cegeka team that is responsible for managing permissions and applications in Microsoft Azure, group chats could be created from the application.

4.1.2 Sending a message

Since application permissions were already in use, an attempt was also made to send messages through the Microsoft Graph API. However, it was later discovered that sending messages with application permissions is not allowed. To send a message in a group chat, a separate entity would be required that can be inserted into the group chat to send the message, such as a Microsoft Teams bot. For the creation of the Microsoft Teams bot, a request to another internal team at Cegeka was required.

However, this request has not been made yet, as project priorities shifted.

4.2 Acceptance environment

The current version of the application contained only two environments: Development and Production. Since the application is moving towards external customers, an acceptance environment is needed to test the latest features without interfering with current processes. This environment is used to test newly developed features. This required an analysis of the existing setup and what needed to be changed.

4.2.1 Current setup

In the existing setup, hardcoded values, values that are directly written in the code instead of being configurable, making them difficult to change and less secure, were used for the origins which were allowed, and which gave API access to certain localhost endpoints in the production environment. Because this poses a security risk, this must be replaced with environment-specific values.

4.2.2 New setup

For the new setup, multiple approaches were researched to set up the CORS policy, CORS (Cross-Origin Resource Sharing) is a security mechanism that controls which external applications are allowed to access the backend API and change the environment links dynamically. It was decided to implement an application-wide CORS configuration using environment variables to configure it dynamically.

4.2.2.1 Method-level CORS

Pros:

- Fine-grained control
- Easy setup

Cons:

- Labor intensive
- Difficult to maintain in large projects

4.2.2.2 Controller-level CORS

Pros:

- Easy separation of controllers
- Easy setup

Cons:

- Multiple controllers may have the same policy
- Difficult to maintain consistency in large projects

4.2.2.3 Application-wide CORS

Pros:

- Centralized → Easy to maintain
- Best option for full APIs

Cons:

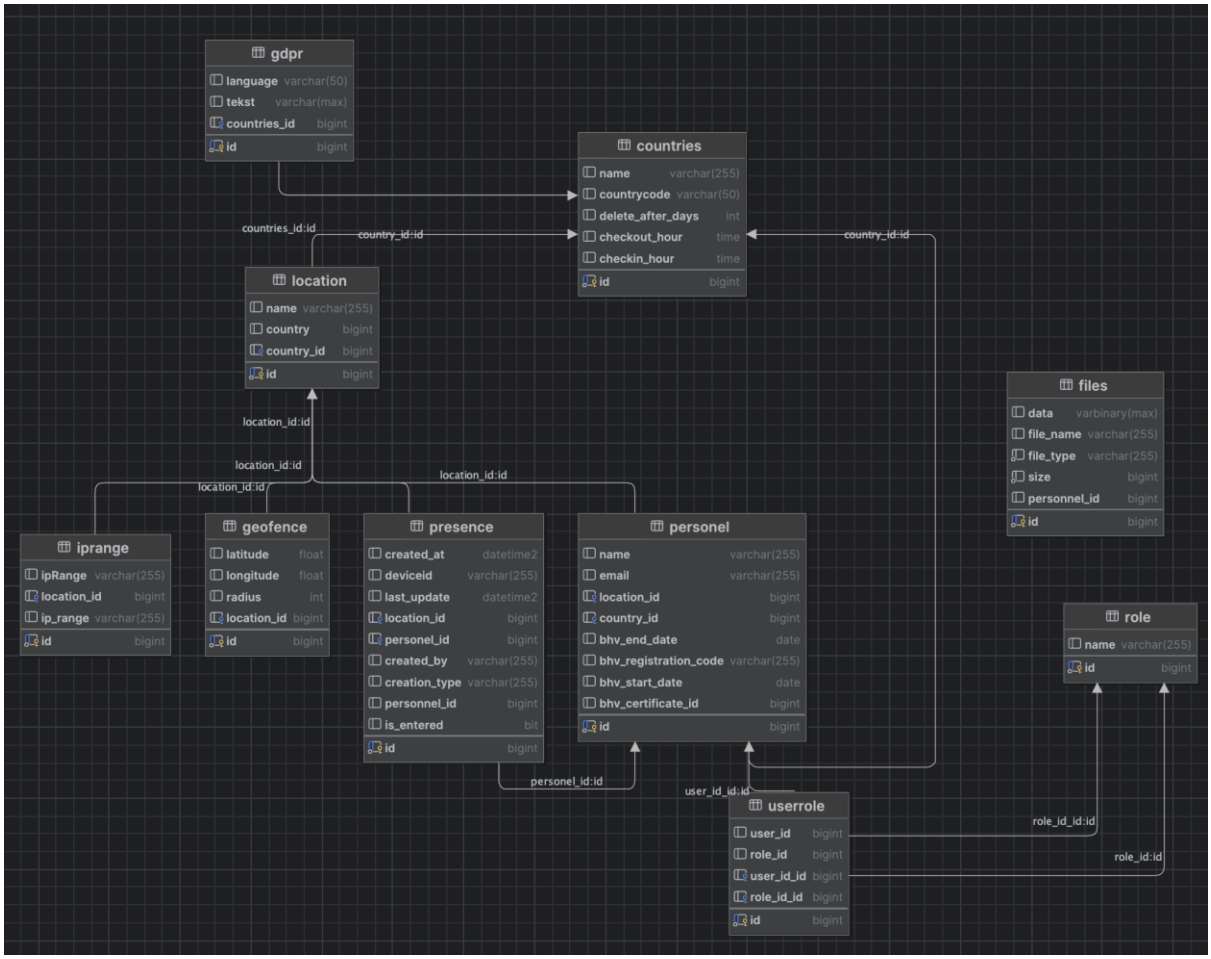
- Less control for specific endpoints

4.3 Database Refactor

A database refactor would be needed in the longer term to review the amount of data that is stored in the database to ensure only the required data would be stored. A simple analysis was made of which data was stored and if it was required and properly named. This refactor will improve performance, reduce redundancy, and make future changes easier.

4.3.1 Current setup

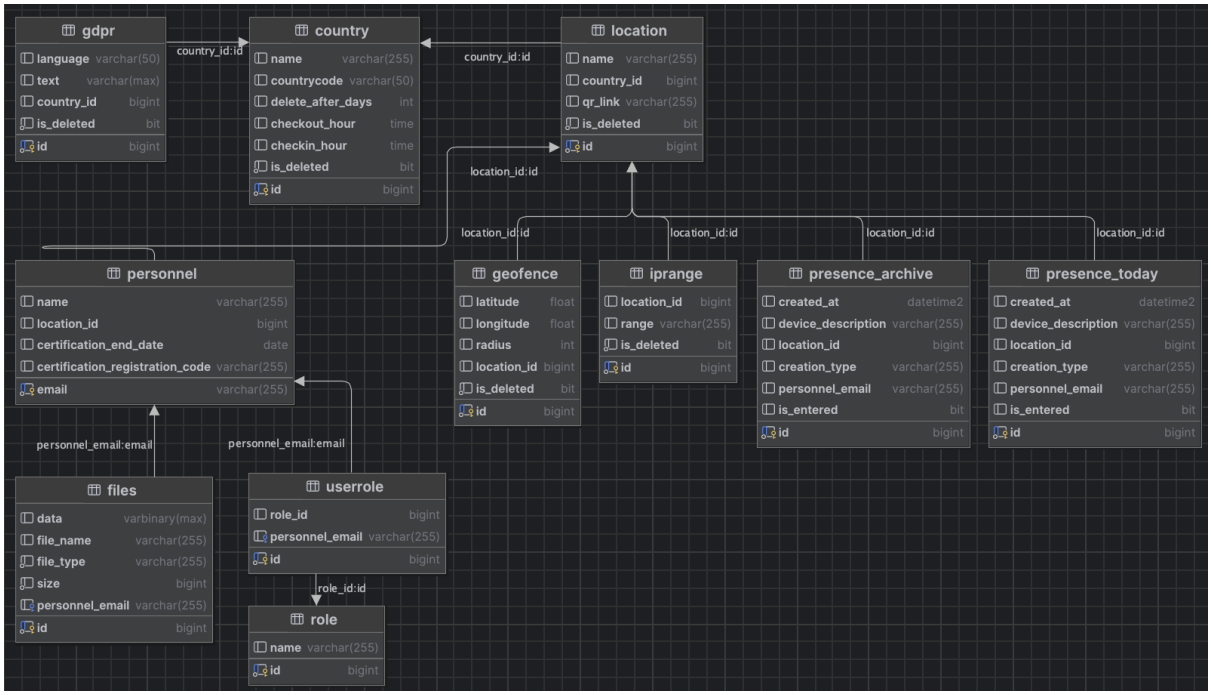
The current situation is a result of many interns working on independent features with almost no focus on naming, duplication of data, optimization, or consistency. This led to many naming errors and unnecessary data.



4.3.2 New setup

The new situation is a result of a migration script created after research into where and how certain data was used and additional future-oriented optimizations. One such optimization was changing the frontend to use “personnel_email” as the primary identifier instead of a numerical ID, in preparation for migrating users to Microsoft Entra.

The research led to the removal of several unnecessary tables. Additional database cleanup is planned as part of the migration to Microsoft Entra.



4.3.3 Migration

For the migration in the production environment, a custom SQL script will be used. To ensure a controlled transition from the original database & API to the refactored version, the migration follows the deployment plan described in Backend deployment plan.

4.4 API Refactor

To make the application future-proof and improve performance and maintainability of the API, refactoring was required. The refactoring of the API happened in three different parts:

1. Modifying the old API to work with the new database, to ensure compatibility for the users with older versions of the application.
2. Conducting research into the desired layout of the new API with Clean Architecture and Domain-Driven Development. This will separate the different parts of the application, so each layer has its own responsibility.
3. Creating the new API.

4.4.1 Current layout

In the current layout, there is no clear structure in place. This makes it difficult to decide where certain new functions or endpoints should be added. It mixes database entities directly with the domain model, which creates a strong coupling between the application logic and the underlying database technology. This will make it more difficult to change in the future that is why the decision was made to implement the clean architecture structure.

com.cegeka.backendproto
- → config
  - – Stores the CORS config file
- → controller
  - – Stores the controller classes of the API
- → domain
  - – Stores the entities classes and DTOs
- → exception
  - – Stores one custom exception
- → repository
  - – Stores repository interfaces
- → services
  - – Stores service interfaces and service implementations
- → tasks
  - – Stores scheduled tasks

4.4.2 New layout

In the new layout, the focus was placed on improving maintainability by implementing a cleaner and more scalable architecture. This structure enforces a clear separation of responsibilities, making it significantly easier to add new features, maintain existing functionality, or modify infrastructure components in the future.

Domain-Driven Development (DDD) was also implemented to ensure a well-structured domain layer. By keeping the domain independent from external frameworks and database technologies, coupling was reduced and the flexibility and maintainability of the application were improved.

com.cegeka.safetysquad
- → api
  - – controller
    - → Stores controller classes
  - – dto
    - → Stores request and response DTOs separated by request and response
  - – exception
    - → Stores an exception handler which handles all custom exceptions
  - – mapper
    - → Stores mapper classes to map from domain model to response DTO
  - – security
- → application
  - – exception
    - → Stores custom exceptions
  - – scheduled
    - → Stores service implementations for scheduled tasks
  - – service
    - → Stores service implementation classes
- → domain
  - – model
    - → Stores pure domain models
  - ⇒ port
    - → Stores repository interfaces
- → infrastructure
  - – config
    - → Stores CORS configuration file.
  - ⇒ persistence
    - → entity
      - – Stores JPA entities
    - → mapper
      - – Stores persistence mappers for conversion between domain model and entity
    - → repository
      - – Stores JpaRepository interface and one implementation for both repository interfaces
  - ⇒ scheduler
    - → Stores scheduled tasks

4.4.3 Creating the new API

During the development of the API itself, the quality and readability of the code was prioritized. This led to research into additional JPA annotations, new methods and libraries. The effects of those improvements are illustrated in the example below.

4.4.3.1 New classes

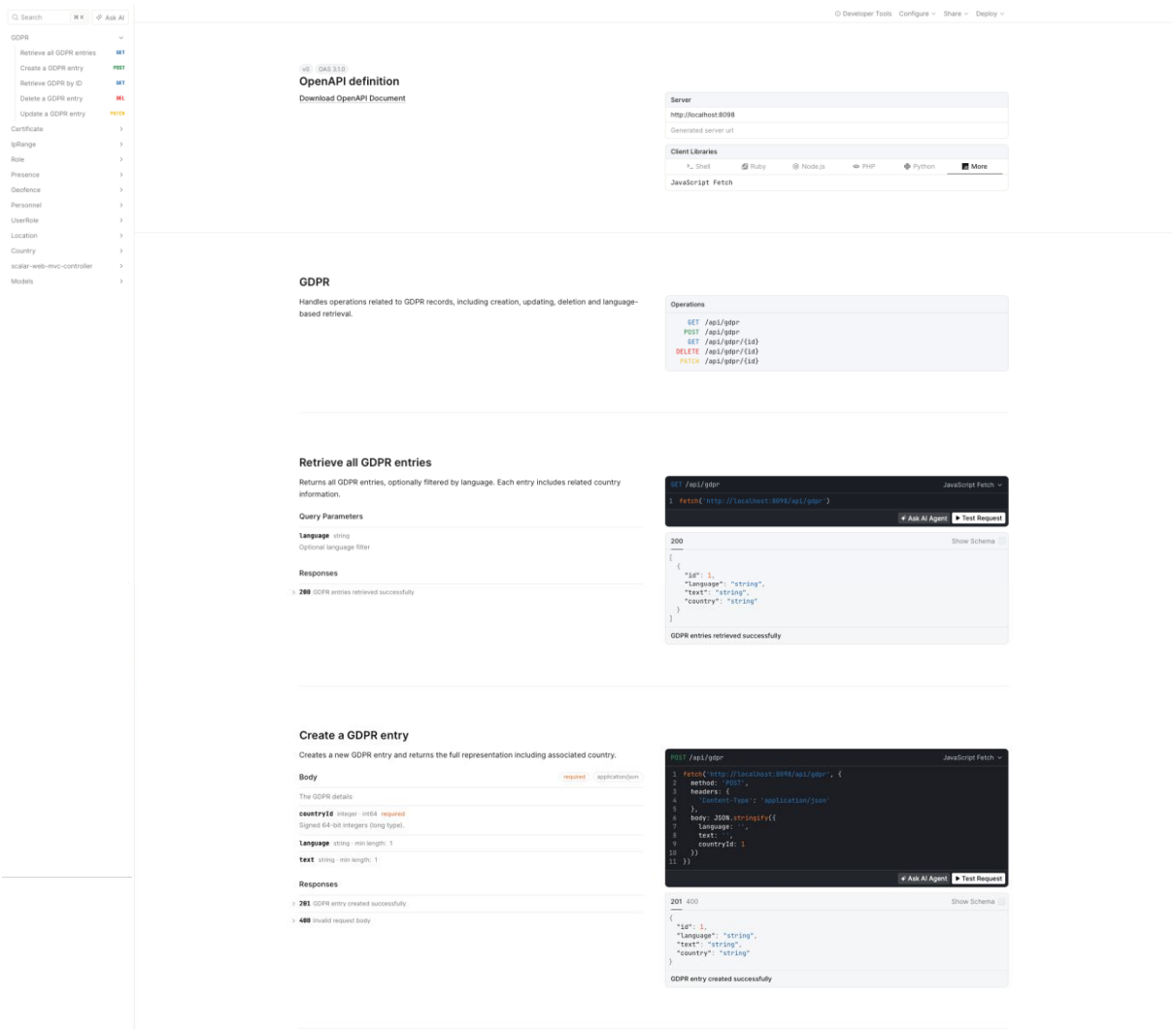
```java
/** REST controller for managing country information. ...*/
@Tag(  Tuur Van Bergen
        name = "Country",
        description = "Manage countries."
)
@RestController
@RequestMapping("/api/countries")
public class CountryController {
    private final CountryService countryService;  6 usages
    private final GdprService gdprService;  2 usages
    private final LocationService locationService;  3 usages
    private final IpRangeService ipRangeService;  2 usages
    private final GeofenceService geofenceService;  2 usages

    private final CountryMapper countryMapper;  7 usages
    private final GdprMapper gdprMapper;  2 usages

    @Autowired  Tuur Van Bergen
    public CountryController(
            CountryService countryService,
            GdprService gdprService,
            LocationService locationService,
            IpRangeService ipRangeService,
            GeofenceService geofenceService,
            CountryMapper countryMapper,
            GdprMapper gdprMapper) {
        this.countryService = countryService;
        this.gdprService = gdprService;
        this.locationService = locationService;
        this.ipRangeService = ipRangeService;
        this.geofenceService = geofenceService;
        this.countryMapper = countryMapper;
        this.gdprMapper = gdprMapper;
    }

    /** Retrieves a list of all countries. ...*/
    @Operation(  Tuur Van Bergen
            summary = "Retrieve all countries",
            description = "Returns a list of all countries."
    )
    @ApiResponses({
            @ApiResponse(responseCode = "200", description = "Country list found"),
    })
    @GetMapping
    public ResponseEntity<List<?>> getAllCountries(
            @Parameter(description = "A boolean value to optionally include locations of each country") @RequestParam(required = false, defaultValue = "false") Boolean locations
    ) {
        var countries = countryService.getAll();
        if (locations) {
            var locationsMap = countries.stream().collect(java.util.stream.Collectors.toMap(Country::id, Country c -> locationService.getAllByCountryId(c.id())));
            var response = countryMapper.toListResponseWithLocations(countries, locationsMap);
            return ResponseEntity.ok(response);
        }

        var response = countryMapper.toListResponse(countries);
        return ResponseEntity.ok(response);
    }

    /** Retrieves information for a specific country. ...*/
    @Operation(  Tuur Van Bergen
            summary = "Retrieve country",
            description = "Returns the info of a country by id."
    )
    @ApiResponses({
            @ApiResponse(responseCode = "200", description = "Country found"),
            @ApiResponse(responseCode = "404", description = "Country not found")
    })
    @GetMapping("/{id}")
    public ResponseEntity<?> getCountryById(
            @Parameter(description = "The id of the country") @PathVariable Long id,
            @Parameter(description = "A boolean value to optionally include locations of the country") @RequestParam(required = false, defaultValue = "false") Boolean location
    ) throws CountryNotFoundException {
        var country = countryService.getById(id);
        if (location) {
            var locations = locationService.getAllByCountryId(country.id());
            List<IpRange> ipRanges = ipRangeService.getAll();
            List<Geofence> geofences = geofenceService.getAll();
            var response = countryMapper.toResponseWithLocations(country, locations, ipRanges, geofences);
            return ResponseEntity.ok(response);
        }

        var response = countryMapper.toResponse(country);
        return ResponseEntity.ok(response);
    }
```

```java
    /** Retrieves gdpr for a specific country. ...*/
    @Operation(  Tuur Van Bergen
            summary = "Retrieve gdpr for country",
            description = "Returns the gdpr of a country by name and language."
    )
    @ApiResponses({
            @ApiResponse(responseCode = "200", description = "Gdpr found"),
            @ApiResponse(responseCode = "404", description = "Gdpr not found for given country and language")
    })
    @GetMapping("/{countryName}/gdpr")
    public ResponseEntity<GdprResponseDto> getGdprByCountryNameAndLanguage(
            @Parameter(description = "The name of the country") @PathVariable String countryName,
            @Parameter(description = "The language for the gdpr") @RequestParam String language
    ) throws GdprNotFoundByLanguageAndCountryException {
        var gdpr = gdprService.getByLanguageAndCountry(language, countryName);
        var response = gdprMapper.toResponse(gdpr);

        return ResponseEntity.ok(response);
    }

    /** Creates country. ...*/
    @Operation(  Tuur Van Bergen
            summary = "Create country",
            description = "Creates a new country."
    )
    @ApiResponses({
            @ApiResponse(responseCode = "201", description = "Country created"),
    })
    @PostMapping()
    public ResponseEntity<CountryResponseDto> createCountry(
            @Parameter(description = "The country details") @Valid @RequestBody CreateCountryRequestDto req
    ) {
        var country = countryService.createCountry(req.name(), req.countryCode(), req.checkinHour(), req.checkoutHour(), req.deleteAfterDays());
        var response = countryMapper.toResponse(country);

        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }
    /** Updates country information. ...*/
    @Operation(  Tuur Van Bergen
            summary = "Patch country",
            description = "Patches the country info by id."
    )
    @ApiResponses({
            @ApiResponse(responseCode = "200", description = "Country patched"),
            @ApiResponse(responseCode = "404", description = "Country not found")
    })
    @PatchMapping("/{id}")
    public ResponseEntity<CountryResponseDto> updateCountry(
            @Parameter(description = "The id of a country") @PathVariable Long id,
            @Parameter(description = "The country details") @RequestBody UpdateCountryRequestDto req
    ) throws CountryNotFoundException {
        var country = countryService.updateCountry(id, req.name(), req.countryCode(), req.checkinHour(), req.checkoutHour(), req.deleteAfterDays());
        var response = countryMapper.toResponse(country);

        return ResponseEntity.status(HttpStatus.OK).body(response);
    }

    /** Delete country information. ...*/
    @Operation(  Tuur Van Bergen
            summary = "Delete country",
            description = "Deletes the country info by id."
    )
    @ApiResponses({
            @ApiResponse(responseCode = "204", description = "Country deleted"),
            @ApiResponse(responseCode = "404", description = "Country not found")
    })
    @DeleteMapping("/{id}")
    public ResponseEntity<?> deleteCountry(
            @Parameter(description = "The id of a country") @PathVariable Long id
    ) throws CountryNotFoundException {
        countryService.deleteCountry(id);

        return ResponseEntity.noContent().build();
    }
}
```

As illustrated in the example class above, all of the controller endpoints are documented using both Javadoc and Swagger, in accordance with the request of the work placement mentor. The Swagger documentation is used to show more detailed explanations about each of these endpoints on the Scalar development frontend used by the project. The Javadoc documentation supports developers by clearly describing the purpose and behavior of each endpoint directly within the codebase.

4.4.4 Scalar

With the creation of the new API, Scalar was selected as the primary tool for API documentation by the work placement mentor. It provides an interactive and developer-friendly interface that improves the accessibility and usability of the API documentation. In addition, it also includes code examples for different frontend libraries, to simplify the implementation of an endpoint into the frontend.



4.5 Backend deployment plan

To ensure a reliable backend deployment, a backend deployment plan was defined. This plan outlines the steps required to deploy the application to the production environment. In addition, multiple fallback scenarios were specified to handle potential errors during the deployment process.

Backend deployment plan

Pre-deployment

- Create a full back-up of the database in azure
- Check if API still has old build artifact

In case anything goes wrong; we reinstate the old database and deploy the old build artifact again the DB script makes breaking changes (not backwards compatible changes) to the API, so both need to be rolled back to previous version.

Deployment

- Disable web services / certain API endpoints to make sure nothing else tries to interact with the database.
- Connect to DB
- Run the update_database.sql script to update database
- Verify script runs without errors; all new fields exist; no existing data is corrupted.

In case verification shows errors, not all fields are created, or existing data is corrupted: Copy the logs, reinstate old database, and fix the script to run without errors.

- Querying the DB to list its FKs
- Build and run the API which will create some new FKs that are needed inside of the database.
- Verify creation of new FKs by querying the DB to list its FKs

In case no new FKs are created, reinstate the old database and deploy old artifact again, update the existing update script and manually create the needed FKs.

Post-deployment

- Check all updated API endpoints
    - There is a postman script for localhost which can be used
- Check logs for exceptions, SQL errors.

4.6 Frontend Refactor

Due to the changes in the API, a frontend refactor was required. This refactor included updates to restore compatibility with the modified API endpoints, as well as a redesign of the frontend pages to improve clarity and overall usability. In addition, the refactoring process involved cleaning up the existing codebase by removing redundant code and improving the file structure.

4.6.1 Impact of API changes on the frontend

Due to changes in the API, all endpoint URLs required modification, as the base API path changed. In addition to these structural changes, the data sent with each request had to be adjusted, and the associated request bodies were modified to align with the updated API request body.

4.6.2 Codebase cleanup and maintainability

During the changing of the frontend, the existing codebase was cleaned by implementing a better file structure, applying naming conventions, simplifying functions and removing duplicate code. These changes enhanced the maintainability of the application. Later, during my internship another intern introduced a full redesign of the frontend as his project was finished.

4.6.3 UI/UX Redesign

This refactor also introduced an opportunity to improve the overall user experience and interface design.

To improve loading performance, unnecessary data rendering and inefficient UI updates were reduced, resulting in faster page load times and more responsive user interactions. In addition, a consistent layout was applied across the whole application to provide the users with a more predictable and uniform interface.

User flows were redefined to group logically related settings within the same context. For example, all location-related properties were consolidated into a single location page. This reduced unnecessary navigation between pages and simplified task execution for the user.

Unnecessary fields were removed from tables, reducing visual clutter. These changes contributed to clearer data presentation and improved focus on the most relevant information. At a later point in the internship a fellow intern had finished his project and conducted a full redesign of the frontend.

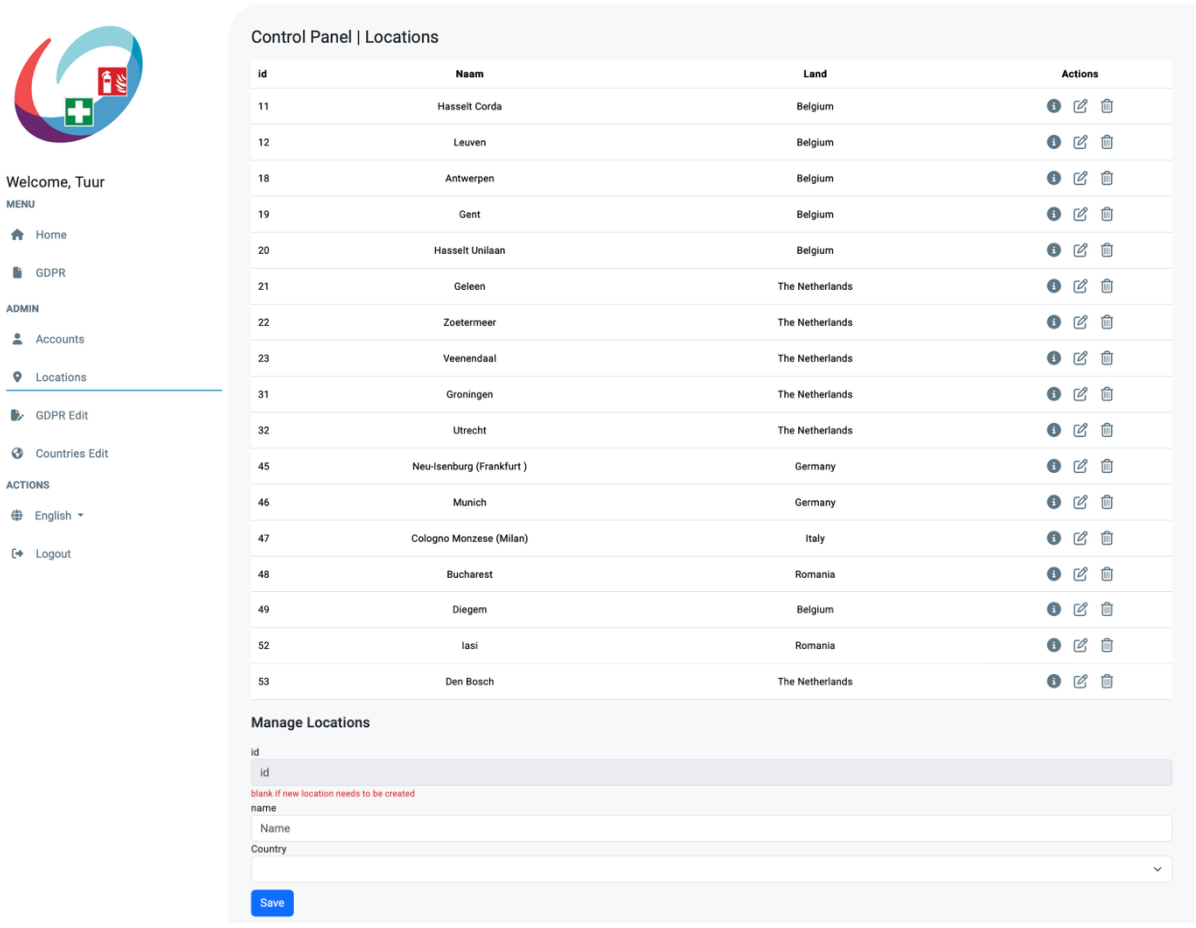
4.6.4 Before-after comparison

The following comparison presents the old and updated versions of a refactored frontend page. This example is included to illustrate the differences between the previous implementation and the improved design following the frontend refactor.

4.6.4.1 Before

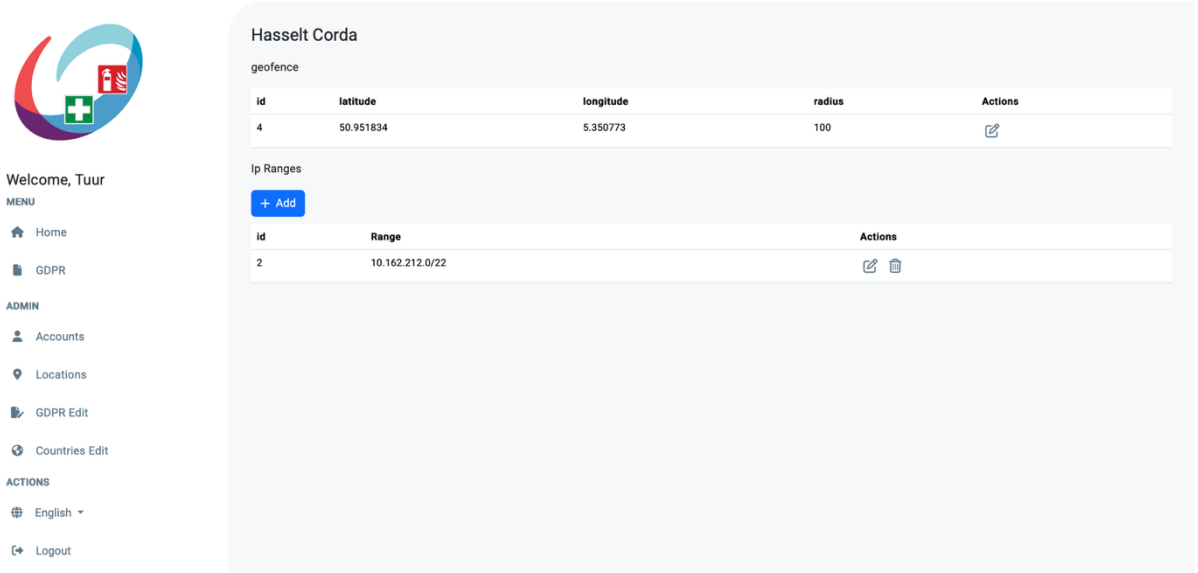
4.6.4.1.1 Locations page

In the original implementation, the Locations page combined multiple interaction patterns. The edit button populated a form at the bottom of the same page based on the selected identifier, while the information button redirected the user to a separate location detail page. This behaviour resulted in inconsistent navigation patterns and reduced clarity, as related configuration options were distributed across different views.



4.6.4.1.2 Locations details page

In the original implementation, the Edit Geofence, Add IP Range, and Edit IP Range actions redirected users to separate pages to perform configuration changes.



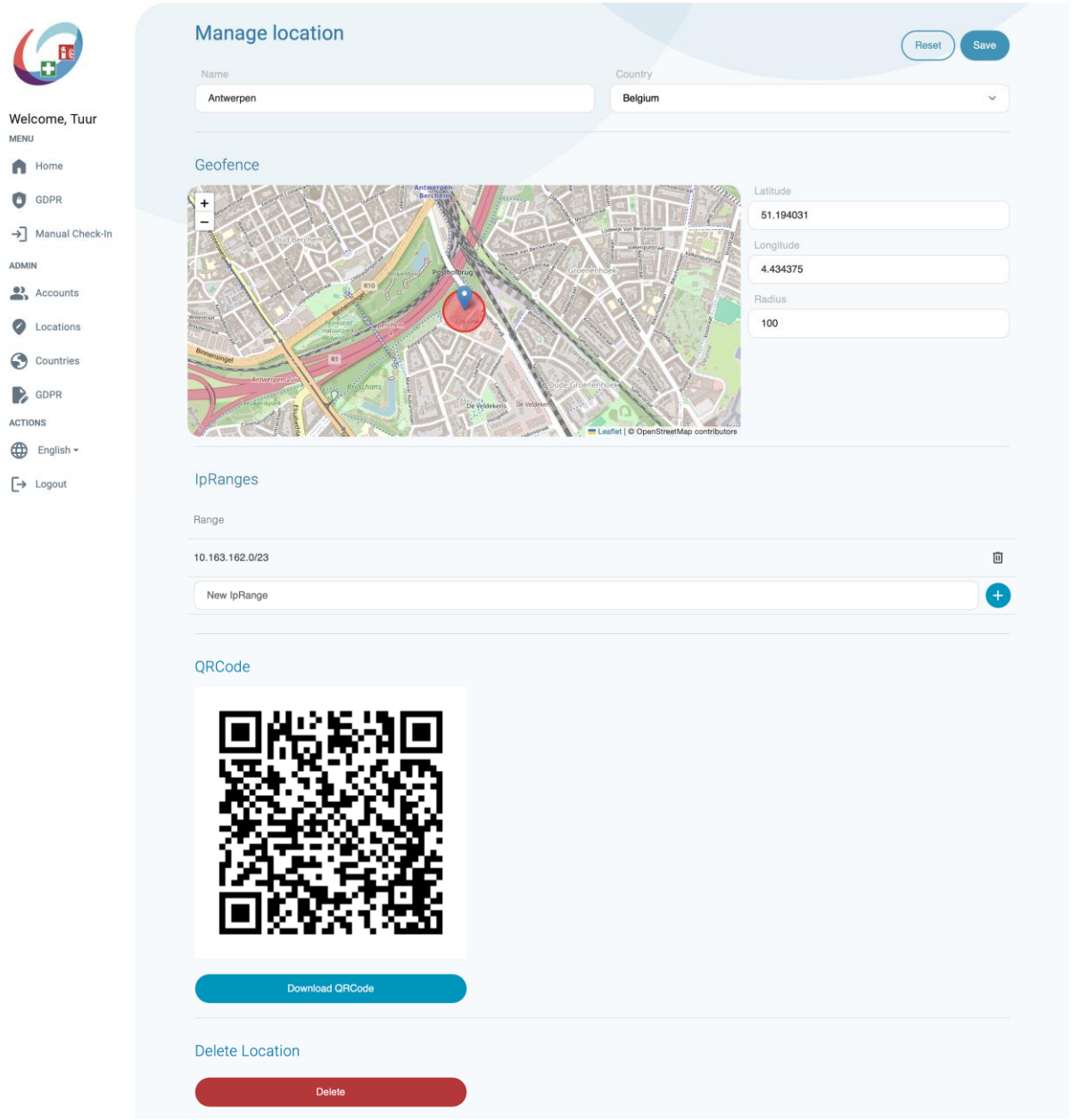
4.6.4.2 After

4.6.4.2.1 Locations page

In the refactored implementation, unnecessary data was removed from the overview page, reducing visual clutter and improving readability. Editing and deletion actions were moved to a dedicated, location-specific page, where all related configuration options are grouped together. This change resulted in a clearer user flow and made location management more intuitive and consistent.

4.6.4.2.2 Location details page

In the refactored version, all location-related details are merged into a single page, where they can be viewed and edited consistently. A map component was added to visually display the configured geofence and by centralizing all configuration options, navigation was simplified and the overall user experience was improved.



4.7 Client Applications Refactor

As a result of the backend refactor, both the desktop and mobile client applications also required changes. This ensured continued compatibility with the updated backend and uninterrupted functionality across all platforms. During this process, a new authentication library was selected for the desktop application, and existing dependencies were updated to improve security. When editing the client applications, we changed the endpoint requests to the new and improved endpoint links.

4.8 Security

For the security of the API and the frontend, the decision was made to implement Microsoft Entra and Role Based Access Control

4.8.1 Microsoft Entra

For the implementation of Microsoft Entra, a cloud-based identity system used to manage users and their access to applications, a lot of new research was required. This research led to a setup which worked with the other internal tools of Cegeka, but also as a standalone application. This way it can be used by other companies that want to use it in the future. The database scheme was refactored again to make sure no unnecessary data was stored in the database.

4.8.1.1 Security Groups

For the assignment of a role to a person, the decision was made to use security groups. These security groups can be used across multiple internal applications for Cegeka. This allows the admin to assign roles to users through the company portal.

4.8.1.2 Standalone application

Due to certain restrictions with internal apps, a request was made for the creation of a standalone app on Azure. This simplified the implementation for the future as it gives permissions to add other Entra's from other companies, this is not allowed in an internal app.

4.8.1.3 App roles

The implementation of a standalone application also made it possible to create app roles for the users. These roles were assigned to our company's security groups, which gave the users from each group the necessary access to our application.

4.8.2 Role Based Access Control

The implementation of Microsoft Entra resulted in the possibility of a better implementation of the Role Based Access Control. Role-Based Access Control (RBAC) ensures that users only have access to features based on their role. As both the frontend and backend had access to these roles directly through a request to Microsoft instead of the frontend needing the backend for role authentication.

4.8.2.1 Frontend

In the frontend, authentication and authorization is done based on the Microsoft Entra. This means a user's data and roles are fetched directly from Microsoft Entra and are being used in the frontend. If a user wants to request data from the API, it sends along the authorization token, this token is subsequently used to check if a user has access to data they are requesting.

4.8.2.2 Backend (API)

Once a request comes in, the backend will authenticate the user that is requesting the data. And will return the data if a user has the correct access permissions. This protects our API from insecure access from outside users.

4.9 Testing

After the backend was completed, tests were added to ensure that the system worked correctly. These tests were focused exclusively on the backend API. The frontend was not included in the testing process, as it was still undergoing frequent changes which would require a lot of changes in the test cases also.

For these tests only unit tests were created. A unit test is used to test a small piece of code, to ensure it behaves as expected when tested on its own.

These tests focused on validating the behaviour of API endpoints and the underlying business logic. This included checking whether the correct responses were returned, whether data was processed correctly, and whether error situations were handled properly.

4.10 Deployment

When the desired features were completed, the decision was made to start the deployment process. This was done in several steps to make sure everything would work.

4.10.1 Deploy to acceptance

For the deploy to acceptance, several changes were required. This deploy was used as a test run for the deployment to production. The deployment plan was used to make sure everything went smoothly, after some changes to the pipeline. The acceptance environment had a successful deploy for both the fronted and backend. This environment was used to test the new frontend and all the client applications; this was done over a period of 2 weeks to make sure everything worked correctly.

4.10.2 Deploy to production

After the testing was completed in the acceptance environment, the decision was made to deploy to production. Before the start of the deploy, the current production environment was taken offline to make sure no requests came in during the breaking changes deployment. The same changes were made to the pipeline as in acceptance to make sure the deploy went smoothly and the deployment plan was followed closely. When the deployment in production was up and running again, a thorough test was performed to make sure everything worked. During this test some small problems were found, these problems were quickly fixed and deployed straight to production. This production has been running smoothly without problems since the 11th of May.

4.11 Presentations

During the internship, two major presentations were conducted to showcase the results and progress of the SafetySquad project. These presentations were prepared together with a business intern who was also assigned to the project. The goal of these presentations was to provide both a technical and functional overview of the application.

4.11.1 Q&A of Social Impact Team

The first presentation was given during a Q&A session attended by the entire Social Impact division. During this presentation, an overview was provided of the SafetySquad application, including its purpose, functionality, and the main improvements implemented during the internship.

In addition to the technical explanation, a business-oriented perspective was included. This covered insights from conducted interviews, the level of market interest, and the remaining steps required before bringing the application to market.

Approximately 125 people attended this session, which provided an opportunity to present the project on a larger scale and gather feedback from the entire team into possible new features.

4.11.2 Management presentation

The second presentation was given to the management team of the Social Impact division. In contrast to the previous presentation, this session focused primarily on the business aspects of the SafetySquad application.

During the presentation, different pricing scenarios were discussed, along with an overview of companies that had already shown interest in the application. In addition, a roadmap was presented outlining the remaining steps required before the application could be brought to market. This included both technical improvements and functional enhancements that would be necessary for a production-ready release.

The presentation provided management with a clear understanding of the potential value of the application and its readiness for commercialization. As a result, the management team requested the creation of a long-term plan, specifically a five-year roadmap, to guide the further development and positioning of the SafetySquad application.

4.12 Multi-tenancy

After the management meeting, the decision was made to continue with the necessary features required for the go-to-market phase. One of these features was ensuring that the application could support multiple tenants. A tenant represents a separate customer or organization within the application, whose data is kept isolated from others.

Supporting multiple tenants is essential for bringing the application to market, as it allows multiple organizations to use the same system while keeping their data separate and secure. This makes the application scalable and suitable for external customers, as new organizations can be added without impacting existing ones.

During the development of the features in this document, multi-tenancy was continuously considered. After implementing the required changes, the application was tested using two different tenants. The results showed that only minor adjustments were needed to fully support this functionality. After applying these final changes, the application functioned correctly in a multi-tenant setup.

4.13 Certificate feature

Another feature that was introduced after the management meeting is the certificate management feature. This feature provides an overview of the expiration dates of all certificates assigned to users within the system.

The main goal of this functionality is to support administrators in managing the validity of first aid and evacuation certificates. By clearly displaying the expiration dates, administrators can easily identify which certificates are close to expiring or have already expired. This allows them to take timely action and ensure that all required personnel maintain valid certifications.

This feature contributes to the overall reliability of the application, as it helps organizations stay compliant with safety requirements by ensuring that qualified personnel are always available.

5 Conclusion

This internship provided me with the opportunity to contribute to the development of the SafetySquad application, with a strong focus on improving the backend architecture. Throughout the internship multiple improvements were realized, like the creation of a new API, an optimization of the database, implementation of Microsoft Entra and much more.

These changes resulted in a system that is ready for future development and possible go-to-market. It improved the quality of the code and provides a clearly defined structure.

Other than the backend, there were also changes to the frontend for a new design and a better internal structure. This contributed to a better user experience and improved maintainability. Furthermore, new features were also added to improve the quality of the system, such as a certificate management page, a dashboard page and several others. In addition, we also updated the client applications, so they work with the new and improved security system that was created for our backend.

The internship also provided me with several research opportunities I got to conduct on my own, such as the implementation of a Teams feature, the implementation of Microsoft Entra and much more. These opportunities provided me with time to learn new things I didn't learn in school.

Overall, this project evolved from an internal tool to a go-to-market ready application that can be expanded upon in the future. Hopefully it will be used by different companies and help them manage their emergency personnel.

6 Reflection

During this internship, I gained valuable experience in both technical and professional aspects of software development. One of the biggest learning opportunities was working on an existing codebase that was already worked on by previous interns. It took some time to understand certain parts of the code or why they made certain choices. After looking into what they created, I identify possible points of improvement before implementing changes. This taught me the importance of writing clean, structured and maintainable code.

When I reflect on the technical part of my internship, I can see that I significantly improved my knowledge of backend development. I learned how to design and implement a well-structured API using different principles such as Clean Architecture and Domain-Driven Development. In addition, I also gained practical experience with database refactoring, Microsoft implementations and deployment processes in a professional environment.

In terms of professional development, I learned how to communicate within a professional team environment, how to give good presentations, and how business decisions are made. This progress was made possible by my mentor who provided me with multiple opportunities to present my research and development and who arranged the other meetings for me.

Finally, I think I also had some professional development. This is mostly shown through the improvement of my communication skills, as I had to do a lot of presentations in both English and Dutch as part of my internship. Working together with a business intern also gave me insight into how technical and business profiles complement each other within a project.

Overall, this internship was a highly valuable experience that strengthened both my technical skills and my ability to work in a professional software development environment. It confirmed my interest in backend development and provided a strong foundation for my future career.